

Assignment 3

TCP & HTTP Implementation over Raw Sockets

Course: COL334 (Computer Networks)

Submitted By

Preksha Rathal : 2024CS50987

Kesar Deval Shah : 2024CS10132

Instructor

Prof. Piyush Kumar Sharma

Department of Computer Science and Engineering
Indian Institute of Technology Delhi

Contents

1 R1: Network Addressing and Routing Configuration **2**

1.1 Topology and Addressing 2

1.2 Configuration Scripts 2

1.3 Hypervisor Network Adapter Configuration (UTM / QEMU) 3

1.4 Configuration Evidence and Verification 3

2 R2: Traceroute and Asymmetric Routing **4**

2.1 Bidirectional Traceroute Results 4

2.2 Explanation of the Asymmetric Paths 5

2.3 Why r2 Appears as 10.10.4.1 5

3 R3: Reverse-Path Filtering **6**

3.1 Modes and Effective Setting 6

3.2 Interface Settings and Justification 6

3.3 Verification Evidence 7

4 R4: Custom TCP Client with a Standard HTTP Server **8**

4.1 Setup 8

4.2 Download and Integrity Verification 8

4.3 Experimental Evidence 8

5 R5: Client–Server Interoperation **9**

5.1 Setup 9

5.2 Results and Integrity Verification 9

5.3 Experimental Evidence 9

6 R6: Packet Capture and TCP Implementation **10**

6.1 TCP Connection Capture 10

6.2 Retransmission and Timer Management 10

6.3 Out-of-Order Buffering and Reassembly 11

6.4 TCP Reno Congestion Control 11

1 R1: Network Addressing and Routing Configuration

1.1 Topology and Addressing

We configured four Linux VMs: client, r1, r2, and server across three LANs and one point-to-point link (Link D). All addresses and routes are configured statically using shell scripts that rely on ip and sysctl.

Forward path

$$\text{Client (10.10.1.10)} \xrightarrow{\text{LAN A}} \text{R1 (10.10.1.1)} \xrightarrow{\text{LAN B}} \text{R2 (10.10.2.2)} \xrightarrow{\text{LAN C}} \text{Server (10.10.3.10)}$$

Return path

$$\text{Server (10.10.3.10)} \rightarrow \text{R2 (10.10.3.1)} \xrightarrow{\text{Link D}} \text{Client (10.10.4.2)}$$

Figure 1: Forward and return paths through the four-VM network.

VM	Network	Address	Forwarding	Role and route constraints
client	LAN A	10.10.1.10/24	0	Routes LAN B and LAN C via 10.10.1.1. Loose filtering on Link D.
client	Link D	10.10.4.2/30	0	Receives the server's return traffic directly from r2.
r1	LAN A	10.10.1.1/24	1	Gateway for the client. Strict reverse-path filtering.
r1	LAN B	10.10.2.1/24	1	Has a single default route via 10.10.2.2.
r2	LAN B	10.10.2.2/24	1	Routes LAN A via 10.10.2.1. Loose filtering on LAN B.
r2	LAN C	10.10.3.1/24	1	Gateway for the server.
r2	Link D	10.10.4.1/30	1	Has a host route for 10.10.1.10 via 10.10.4.2 and no default route.
server	LAN C	10.10.3.10/24	0	Default route via 10.10.3.1. Strict reverse-path filtering.

Table 1: Static IPv4 addressing and forwarding configuration.

1.2 Configuration Scripts

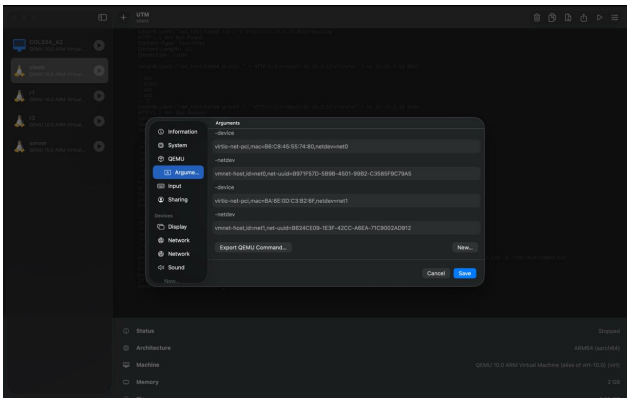
The scripts in netcfg/ configure addresses, routes, forwarding, and reverse-path filtering:

- **client.sh:** Configures both interfaces, disables forwarding, routes LAN B/C via 10.10.1.1, and enables loose filtering on Link D.
- **r1.sh:** Configures LAN A/B, enables forwarding, sets the default route via 10.10.2.2, and retains strict filtering.
- **r2.sh:** Configures LAN B/C and Link D, enables forwarding, routes LAN A via 10.10.2.1 and the client host via 10.10.4.2, and has no default route. Filtering is loose on LAN B.
- **server.sh:** Configures 10.10.3.10/24, disables forwarding, sets the default route via 10.10.3.1, and retains strict filtering.

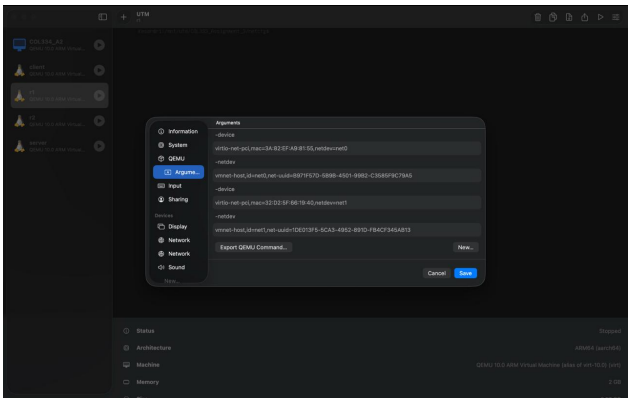
1.3 Hypervisor Network Adapter Configuration (UTM / QEMU)

In UTM on Apple Silicon macOS, isolated internal networks are provisioned using isolated point-to-point network devices without host adapters or DHCP servers. Each virtual machine's network links are configured through UTM's QEMU Arguments pane with explicit -device virtio-net-pci and corresponding -netdev specifications matching the assigned network interfaces.

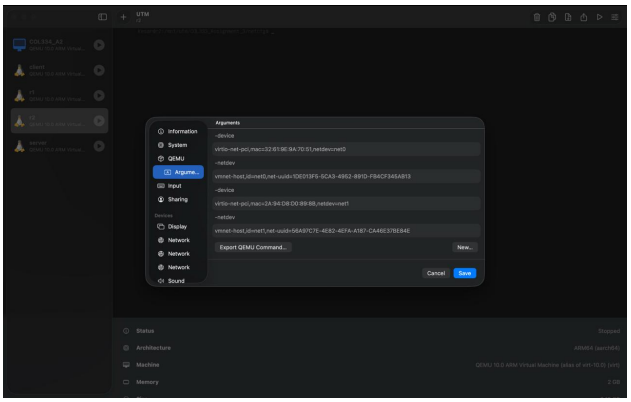
Client VM



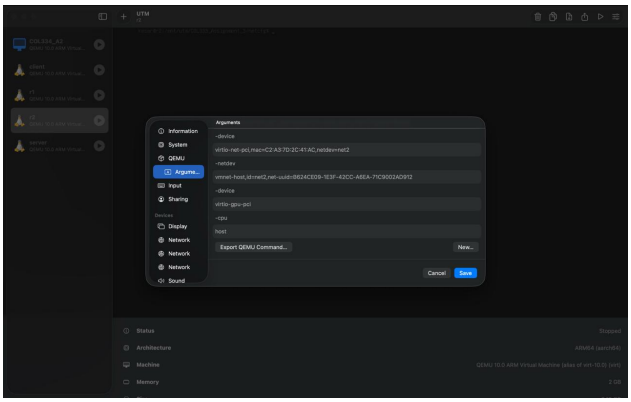
Router R1 VM



Router R2 VM (Part 1)



Router R2 VM (Part 2)



Server VM

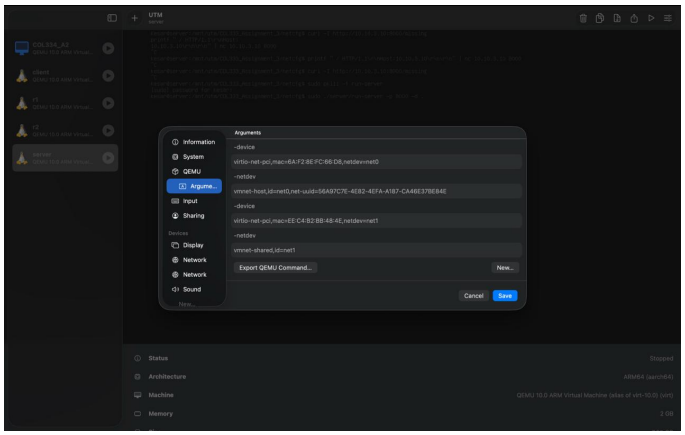


Figure 2: UTM QEMU Arguments settings showing `-device` and `-netdev` adapter configurations for all four virtual machines.

1.4 Configuration Evidence and Verification

We ran the configuration scripts and checked interface addresses, routes, and link states on both laptop environments.



Figure 3: Address assignments and routing-table evidence from both laptop environments.

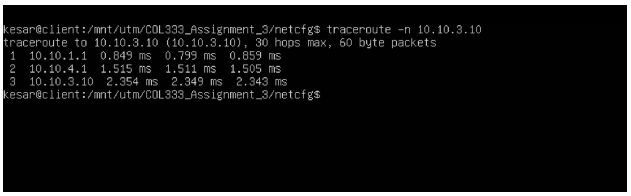
Connectivity was verified with ping and traceroute. The client-to-server trace follows client → r1 → r2 → server; r2 responds using its Link D address, 10.10.4.1, because the probes reach it over that interface. The server-to-client trace follows server → r2 → client, using Link D for the final hop. These results confirm the intended asymmetric paths.

2 R2: Traceroute and Asymmetric Routing

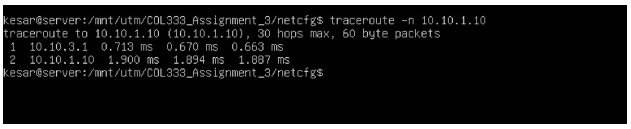
2.1 Bidirectional Traceroute Results

We ran traceroute -n between the client and server on both laptop environments. Both showed the expected asymmetric paths: three hops from client to server and two hops in the reverse direction.

Laptop 1

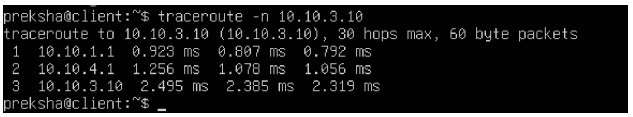


(a) Client → Server (3 hops)

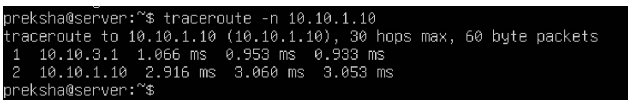


(b) Server → Client (2 hops)

Laptop 2



(a) Client → Server (3 hops)



(b) Server → Client (2 hops)

Figure 4: Bidirectional traceroute results from both laptop environments.

Hop	Client → Server	Server → Client	Explanation
1	10.10.1.1 (r1)	10.10.3.1 (r2)	The first router reached from each endpoint.
2	10.10.4.1 (r2)	10.10.1.10 (client)	Forward probes reach r2 over Link D; reverse traffic reaches the client directly over Link D.
3	10.10.3.10 (server)	.	The forward trace reaches the server after r1 and r2.

Table 2: Observed hops in both directions.

2.2 Explanation of the Asymmetric Paths

The client routes traffic for the server's subnet, 10.10.3.0/24, through r1. R1 forwards it to r2 using its default route via 10.10.2.2, and r2 delivers it to the server over LAN C. The forward path is therefore:

$$\text{Client} \rightarrow \text{r1} \rightarrow \text{r2} \rightarrow \text{Server}.$$

For return traffic, the server uses its default route via 10.10.3.1 to reach r2. R2 has a specific host route for 10.10.1.10/32 via 10.10.4.2, which takes precedence over its broader route for LAN A via r1. It therefore sends packets directly to the client over Link D, bypassing r1:

$$\text{Server} \rightarrow \text{r2} \rightarrow \text{Client}.$$

2.3 Why r2 Appears as 10.10.4.1

In the client-to-server trace, r2 appears as 10.10.4.1 rather than 10.10.2.2. Traceroute probes arrive at r2 over LAN B, but r2's route back to the probe source, 10.10.1.10, uses Link D via 10.10.4.2. When r2 generates an ICMP Time Exceeded response, its source address is selected from the outgoing interface, Link D (10.10.4.1). The client therefore reports 10.10.4.1 as the second hop.

3 R3: Reverse-Path Filtering

3.1 Modes and Effective Setting

Linux reverse-path filtering (rp_filter) validates packet source addresses against the routing table:

- **0: Disabled:** No reverse-path validation.
- **1: Strict:** Drops a packet if the best reverse route to its source uses a different interface from the one on which it arrived.
- **2: Loose:** Accepts a packet if its source is reachable through any interface.

The effective setting is the maximum of the global and interface-specific values:

$$\text{effective rp_filter} = \max(conf.all.rp_filter, conf.IF.rp_filter).$$

All VMs start with strict filtering enabled.

3.2 Interface Settings and Justification

Machine	Interface	Mode	Reason
client	LAN A	1 (Strict)	Traffic on the client LAN follows the expected route.
client	Link D	2 (Loose)	Server replies arrive on Link D, although the route back to the server uses LAN A.
r1	All	1 (Strict)	Forwarding between LAN A and LAN B is compatible with strict filtering.
r2	LAN B	2 (Loose)	Client packets arrive via r1 on LAN B, but the route back to the client uses Link D.
r2	LAN C, Link D	1 (Strict)	Strict filtering is retained on the other interfaces.
server	LAN C	1 (Strict)	Server traffic uses LAN C through gateway 10.10.3.1.

Table 3: Reverse-path filtering configuration and rationale.

Loose filtering is required at two points because the forward and return paths differ:

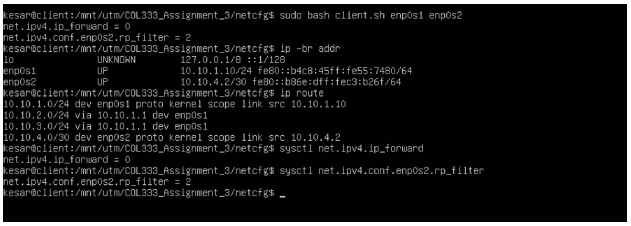
- **Client, Link D:** Replies from server (10.10.3.10) arrive on Link D, but the client's route to that source uses LAN A via r1. Strict filtering would reject these replies; loose filtering permits them because the source remains reachable.
- **r2, LAN B:** Packets from client (10.10.1.10) arrive through r1 on LAN B, while r2's specific host route to the client uses Link D via 10.10.4.2. Strict filtering would reject these packets; loose filtering allows them.

R1 and the server retain strict filtering, as their required traffic paths are compatible with it. This preserves the specified filtering policy while allowing the asymmetric routes.

3.3 Verification Evidence

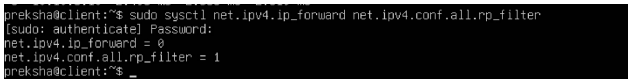
We verified IP forwarding and reverse-path filtering on both laptop environments. The screenshots below show the relevant sysctl values and, where available, interface-specific overrides.

Laptop 1



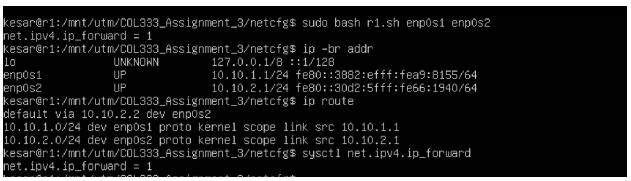
(a) Client: forwarding disabled; Link D filtering set to loose mode.

Laptop 2



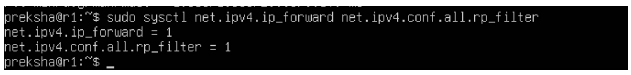
(b) Client: forwarding disabled; global reverse-path filtering set to strict mode.

Laptop 1



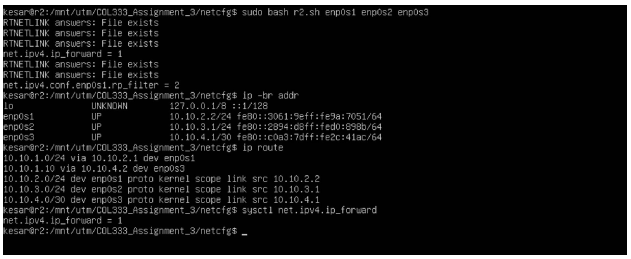
(a) r1: forwarding enabled; default route via 10.10.2.2.

Laptop 2



(b) r1: forwarding enabled; global reverse-path filtering set to strict mode.

Laptop 1



(a) r2: forwarding enabled; loose filtering on LAN B.

Laptop 2



(b) r2: forwarding enabled; global reverse-path filtering set to strict mode.

Laptop 1



(a) Server: forwarding disabled; strict filtering configured on the interface.

Laptop 2



(b) Server: forwarding disabled; global reverse-path filtering set to strict mode.

The screenshots confirm the required forwarding settings: forwarding is disabled on the client and server and enabled on r1 and r2. They also show the global strict-filter setting and the loose interface-specific overrides where captured. Interface-specific values not visible in a screenshot should be verified separately before claiming they were confirmed on both laptops.

4 R4: Custom TCP Client with a Standard HTTP Server

4.1 Setup

We tested the custom HTTP client against Python's standard HTTP server, running on the server VM at 10.10.3.10:8000. The server provided two files:

- **output.html**: 22 bytes.
- **large_file.bin**: 150 KB (153,600 bytes), generated using /dev/urandom.

For raw TCP socket interoperability, GRO was disabled on the relevant interface and kernel-generated TCP RST packets were filtered using iptables.

4.2 Download and Integrity Verification

We downloaded both objects using the custom client and curl. The downloads completed with exit code 0, and the MD5 hashes matched between the original files and the downloaded copies.

File	Size	Exit Code	MD5 Checksum
output.html	22 B	0	ad92f9a5ee8aee56f44ac52d8096afe0
large_file.bin	153,600 B	0	2779b51746afc991dd6886b1fa1ffaed

Table 4: Downloaded file sizes, exit codes, and verified MD5 checksums.

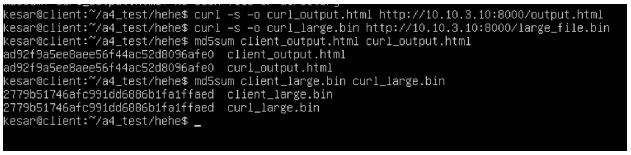
4.3 Experimental Evidence

Figures 4.3 and 4.3 show the download results and server logs from both laptop environments.

Laptop 1

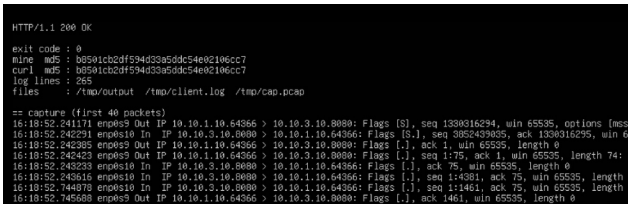


(a) Client download results

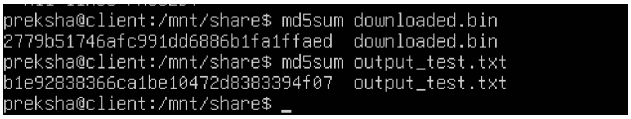


(b) Curl and MD5 comparison

Laptop 2

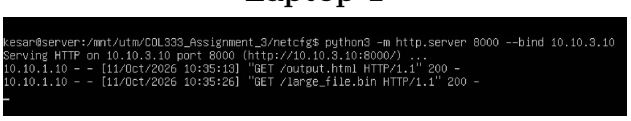


(a) Client download results



(b) MD5 verification

Laptop 1



Laptop 2



The matching MD5 checksums confirm that the downloaded files are byte-for-byte identical to the originals. The server logs provide additional evidence that the requests reached the server and were handled.

5 R5: Client–Server Interoperation

5.1 Setup

We tested the custom HTTP server on 10.10.3.10:8080 using three clients: curl, wget, and our custom HTTP client. Each downloaded large_file.bin (153,600 bytes) from the server.

5.2 Results and Integrity Verification

All three downloads completed successfully. Their MD5 checksums matched the original file, confirming byte-for-byte integrity.

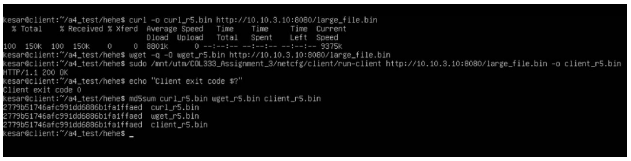
Client	Output file	MD5 checksum
Server original	large_file.bin	2779b51746afc991dd6886b1fa1ffaed
curl	curl_r5.bin	2779b51746afc991dd6886b1fa1ffaed
wget	wget_r5.bin	2779b51746afc991dd6886b1fa1ffaed
Custom client	client_r5.bin	2779b51746afc991dd6886b1fa1ffaed

Table 5: MD5 checksums of the original file and all downloaded copies.

5.3 Experimental Evidence

The following screenshots show the client-side downloads and checksum comparisons, together with the server-side execution, on both laptop environments.

Laptop 1

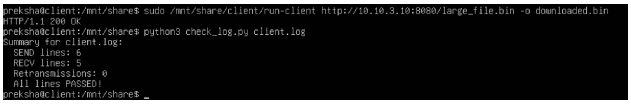


(a) Client downloads and MD5 results



(b) Custom server and original file checksum

Laptop 2



(a) Custom client download



(b) Client checksum comparison

Figure 5: Interoperation and integrity verification in both laptop environments.

Matching checksums demonstrate that the downloaded contents are identical to the original file. The results verify interoperation between standard HTTP clients and the custom server, as well as successful use of the custom client, provided each recorded transfer completed successfully.

6 R6: Packet Capture and TCP Implementation

6.1 TCP Connection Capture

We captured the custom client's download of output.html using tcpdump -nn. The 11 packets show the TCP handshake, HTTP request and response, and orderly connection teardown.

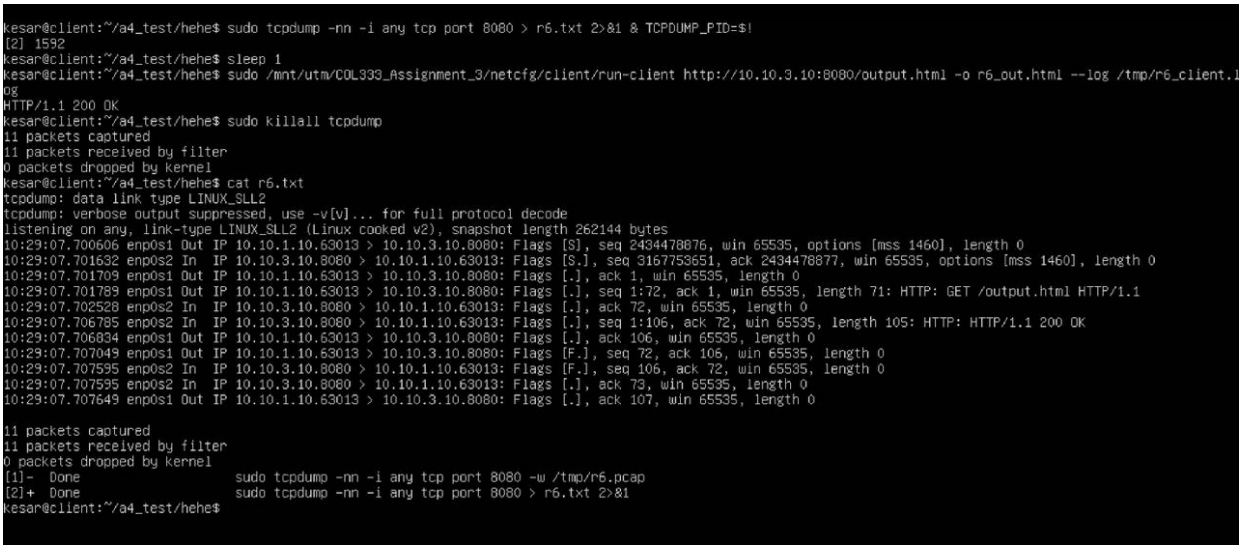


Figure 6: Packet capture of the custom client's HTTP request to the custom server.

Packets	Direction and flags	Purpose
1-3	Client ↔ Server; SYN, SYN-ACK, ACK	Complete the three-way handshake. The SYN-ACK arrives on the client's Link D interface.
4-5	Client → Server; PSH-ACK, ACK	Send the HTTP GET request and acknowledge its receipt.
6-7	Server → Client; PSH-ACK, ACK	Return the HTTP response (105 bytes, including headers and the 22-byte body) and acknowledge it.
8-11	Client ↔ Server; FIN-ACK, ACK	Close the connection cleanly using FIN and ACK packets.

Table 6: Summary of the 11-packet TCP connection.

The client sends its SYN over LAN A, while the SYN-ACK returns over Link D. This is consistent with the configured asymmetric routing. The exchange then completes the HTTP transfer and closes the connection without a reset in the captured sequence.

6.2 Retransmission and Timer Management

The TCP engine tracks unacknowledged segments in an in-flight queue and retransmits them when their retransmission timeout (RTO) expires. RTT samples from eligible, non-retransmitted segments are used to update the smoothed RTT and RTT variance according to RFC 6298 and Karn's algorithm:

$$\begin{aligned}\mathrm{RTTVAR} &\leftarrow 0.75\,\mathrm{RTTVAR} + 0.25\,|\mathrm{SRTT} - \mathrm{Sample}| \\ \mathrm{SRTT} &\leftarrow 0.875\,\mathrm{SRTT} + 0.125\,\mathrm{Sample} \\ \mathrm{RTO} &\leftarrow \min(1.0, \max(0.1, \mathrm{SRTT} + 4\,\mathrm{RTTVAR}))\,\mathrm{s}\end{aligned}$$

The implementation caps the RTO at 1.0 s. On timeout, the oldest unacknowledged segment is retransmitted and the timeout is backed off exponentially. Handshake and teardown timers separately enforce the required SYN and FIN retry deadlines.

6.3 Out-of-Order Buffering and Reassembly

The receive path maintains the next expected sequence number, `rcv_nxt`. Segments arriving ahead of this position are buffered by sequence number if they fall within the receive window. When missing data arrives, contiguous buffered segments are drained into the receive stream and `rcv_nxt` advances. Duplicate or overlapping data is trimmed as needed, and cumulative ACKs report the next expected sequence number.

Sequence comparisons account for 32-bit wraparound using modular arithmetic. These mechanisms preserve data order when segments arrive out of order.

6.4 TCP Reno Congestion Control

The sender implements TCP Reno using the congestion window (`cwnd`) and slow-start threshold (`ssthresh`):

- **Slow start:** While `cwnd` is below `ssthresh`, it grows by up to one MSS per newly acknowledged segment, producing approximately exponential growth per RTT.
- **Congestion avoidance:** Once `cwnd` reaches `ssthresh`, additive increase grows the window by approximately one MSS per RTT:

$$\mathrm{cwnd} \leftarrow \mathrm{cwnd} + \frac{\mathrm{MSS}^2}{\mathrm{cwnd}}$$

- **Fast retransmit and recovery:** On the third duplicate ACK, the sender retransmits the oldest unacknowledged segment and reduces its sending rate by updating `ssthresh` and `cwnd`. Additional duplicate ACKs inflate `cwnd`; a new cumulative ACK ends recovery.
- **Timeout:** On RTO expiry, the sender reduces `ssthresh`, resets `cwnd` to one MSS, and returns to slow start.

The packet capture verifies the normal connection lifecycle. Retransmission, out-of-order recovery, and congestion-control responses are implementation mechanisms and require suitable loss or reordering tests to demonstrate their behavior experimentally.